

Sprint 10 — Chain of Responsibility Pattern

Class 1

Learning Objectives

By the end of this class, we should be able to:

- Explain why Chain of Responsibility exists.
- Recognize long conditional pipelines.
- Understand Handler, Concrete Handler, and Next Handler.
- Build a processing chain.
- Stop or continue the chain correctly.
- Connect the pattern to SOLID.
- Know when Chain of Responsibility is better than one giant service.

Lesson 1 — Why Chain of Responsibility Exists

Imagine we are building a payment API.

Before processing payment, we need to:

1. Validate request
2. Authenticate user
3. Authorize permission
4. Check rate limit
5. Check fraud
6. Process payment

A beginner may put everything in one service.

```
public class PaymentService {  
  
    public void pay(PaymentRequest request) {  
        validateRequest(request);  
        authenticate(request);  
        authorize(request);  
        checkRateLimit(request);  
        checkFraud(request);  
        processPayment(request);  
    }  
}
```

At first, this looks fine.

But requirements grow.

Lesson 2 — The Problem

Tomorrow business says: Add blacklist check.

Then: Add transaction limit check.

Then: Add country restriction.

Then: Add maintenance-mode check.

Now:

```
public void pay(PaymentRequest request) {  
  
    validateRequest(request);  
    authenticate(request);  
    authorize(request);  
    checkBlacklist(request);  
    checkRateLimit(request);  
}
```

```
checkTransactionLimit(request);

checkCountryRestriction(request);

checkFraud(request);

checkMaintenanceMode(request);

processPayment(request);
}
```

The service becomes a pipeline manager for everything.

What Problems Do We See?

- Too many responsibilities
- Difficult to reorder checks
- Difficult to reuse checks
- Difficult to test checks independently
- Hard to enable or disable one step
- Adding checks changes the same class repeatedly

This is where Chain of Responsibility helps.

Lesson 3 — The Core Idea

Instead of:

```
PaymentService
—— validate
—— authenticate
—— authorize
```

—— rateLimit
—— fraud
—— payment

we build:

```
Request
|
ValidationHandler
|
AuthenticationHandler
|
AuthorizationHandler
|
RateLimitHandler
|
FraudHandler
|
PaymentHandler
```

Each handler knows only:

- Its own responsibility
- The next handler

Key Definition

Chain of Responsibility passes a request through a sequence of handlers, allowing each handler to process it, reject it, or pass it to the next handler.

Each handler does one job, then decides whether the request should continue.

Lesson 4 — Handler Interface

```
public interface PaymentHandler {  
  
    void handle(PaymentContext context);  
  
    void setNext(PaymentHandler next);  
}
```

The chain needs two things:

```
handle(...)  
setNext(...)
```

Lesson 5 — Base Handler

Instead of repeating next-handler logic everywhere, create a base class.

```
public abstract class BasePaymentHandler  
    implements PaymentHandler {  
  
    private PaymentHandler next;  
  
    @Override  
    public void setNext(PaymentHandler next) {  
        this.next = next;  
    }  
  
    protected void next(PaymentContext context) {  
  
        if (next != null) {  
            next.handle(context);  
        }  
    }  
}
```

```
}
```

Concrete handlers only focus on their own rule.

Lesson 6 — Validation Handler

```
public class ValidationHandler
    extends BasePaymentHandler {

    @Override
    public void handle(PaymentContext context) {

        if (context.request() == null) {
            throw new IllegalArgumentException(
                "Payment request is required"
            );
        }

        if (context.request().amount() == null) {
            throw new IllegalArgumentException(
                "Amount is required"
            );
        }

        System.out.println(
            "Validation passed"
        );

        next(context);
    }
}
```

Lesson 7 — Authentication Handler

```

public class AuthenticationHandler
    extends BasePaymentHandler {

    @Override
    public void handle(PaymentContext context) {

        if (context.userId() == null
            || context.userId().isBlank()) {

            throw new SecurityException(
                "User is not authenticated"
            );
        }

        System.out.println(
            "Authentication passed"
        );

        next(context);
    }
}

```

Lesson 8 — Authorization Handler

```

public class AuthorizationHandler
    extends BasePaymentHandler {

    @Override
    public void handle(PaymentContext context) {

        if (!context.permissions()
            .contains("PAYMENT_CREATE")) {

```

```

        throw new SecurityException(
            "User has no payment permission"
        );
    }

    System.out.println(
        "Authorization passed"
    );

    next(context);
}
}

```

Lesson 9 — Rate Limit Handler

```

public class RateLimitHandler
    extends BasePaymentHandler {

    @Override
    public void handle(PaymentContext context) {

        if (context.requestsToday() >= 100) {

            throw new IllegalStateException(
                "Daily payment limit reached"
            );
        }

        System.out.println(
            "Rate limit passed"
        );

        next(context);
    }
}

```



```
}  
}
```

Lesson 10 — Fraud Handler

```
import java.math.BigDecimal;  
  
public class FraudHandler  
    extends BasePaymentHandler {  
  
    @Override  
    public void handle(PaymentContext context) {  
  
        if (context.request()  
            .amount()  
            .compareTo(  
                new BigDecimal("10000")  
            ) > 0) {  
  
            throw new IllegalStateException(  
                "Payment requires manual review"  
            );  
        }  
  
        System.out.println(  
            "Fraud check passed"  
        );  
  
        next(context);  
    }  
}
```

Lesson 11 — Final Payment Handler

The last handler performs the real operation.

```
public class ProcessPaymentHandler
    extends BasePaymentHandler {

    @Override
    public void handle(PaymentContext context) {

        System.out.println(
            "Processing payment: "
            + context.request().amount()
        );

        context.markProcessed();
    }
}
```

Notice:

No need to call: `next(context);`

because this is the last handler.

Lesson 12 — Payment Context

```
import java.util.Set;

public class PaymentContext {

    private final PaymentRequest request;
```

```
private final String userId;

private final Set<String> permissions;

private final int requestsToday;

private boolean processed;

public PaymentContext(
    PaymentRequest request,
    String userId,
    Set<String> permissions,
    int requestsToday
) {
    this.request = request;
    this.userId = userId;
    this.permissions = permissions;
    this.requestsToday = requestsToday;
}

public PaymentRequest request() {
    return request;
}

public String userId() {
    return userId;
}

public Set<String> permissions() {
    return permissions;
}

public int requestsToday() {
    return requestsToday;
}

public boolean processed() {
```

```
        return processed;
    }

    public void markProcessed() {
        this.processed = true;
    }
}
```

Lesson 13 — Build the Chain

```
PaymentHandler validation = new ValidationHandler();
```

```
PaymentHandler authentication = new AuthenticationHandler();
```

```
PaymentHandler authorization = new AuthorizationHandler();
```

```
PaymentHandler rateLimit = new RateLimitHandler();
```

```
PaymentHandler fraud =new FraudHandler();
```

```
PaymentHandler payment = new ProcessPaymentHandler();
```

Connect:

```
validation.setNext(authentication);
```

```
authentication.setNext(authorization);
```

```
authorization.setNext(rateLimit);
```

```
rateLimit.setNext(fraud);
```

```
fraud.setNext(payment);
```

Now:

```
validation.handle(context);
```

One call starts the entire chain.

Execution

```
Validation
|
Authentication
|
Authorization
|
Rate Limit
|
Fraud
|
Payment
```

Lesson 14 — Stop the Chain

This is critical.

Suppose authentication fails.

```
if (context.userId() == null) {

    throw new SecurityException(
        "Unauthenticated"
    );
}
```

Then:

Validation (pass) -> Authentication (fail) -> STOP

Authorization never runs.

Payment never runs.

Another Style — Handle and Stop Without Exception

Not every chain uses exceptions.

Example approval workflow:

```
public interface ExpenseHandler {  
  
    boolean handle(ExpenseRequest request);  
}
```

Manager:

```
public boolean handle(  
    ExpenseRequest request  
) {  
  
    if (request.amount()  
        .compareTo(  
            new BigDecimal("1000")  
        ) <= 0) {  
  
        approve(request);  
    }  
}
```

```
        return true;
    }

    return false;
}
```

If Manager cannot handle it, request goes to Director.

Lesson 15 — Approval Example

This is the classic business example.

Expense Request -> Team Lead -> Manager -> Director -> CEO

Limits:

Team Lead $\leq$ \$500

Manager $\leq$ \$2,000

Director $\leq$ \$10,000

CEO $>$ \$10,000

Handler

```
public abstract class ExpenseApprover {

    private ExpenseApprover next;

    public ExpenseApprover setNext(
        ExpenseApprover next
    ) {
        this.next = next;
    }
}
```

```

        return next;
    }

    public void approve(
        ExpenseRequest request
    ) {

        if (canApprove(request)) {

            doApprove(request);

            return;
        }

        if (next != null) {

            next.approve(request);

            return;
        }

        throw new IllegalStateException(
            "No approver available"
        );
    }

    protected abstract boolean canApprove(
        ExpenseRequest request
    );

    protected abstract void doApprove(
        ExpenseRequest request
    );
}

```

Team Lead


```

import java.math.BigDecimal;

public class TeamLeadApprover
    extends ExpenseApprover {

    @Override
    protected boolean canApprove(
        ExpenseRequest request
    ) {

        return request.amount()
            .compareTo(
                new BigDecimal("500")
            ) <= 0;
    }

    @Override
    protected void doApprove(
        ExpenseRequest request
    ) {

        System.out.println(
            "Approved by Team Lead"
        );
    }
}

```

Manager

```

import java.math.BigDecimal;

public class ManagerApprover

```

```

        extends ExpenseApprover {

    @Override
    protected boolean canApprove(
        ExpenseRequest request
    ) {

        return request.amount()
            .compareTo(
                new BigDecimal("2000")
            ) <= 0;
    }

    @Override
    protected void doApprove(
        ExpenseRequest request
    ) {

        System.out.println(
            "Approved by Manager"
        );
    }
}

```

Director

```

import java.math.BigDecimal;

public class DirectorApprover
    extends ExpenseApprover {

    @Override
    protected boolean canApprove(
        ExpenseRequest request
    ) {

```

```

    ) {

        return request.amount()
            .compareTo(
                new BigDecimal("10000")
            ) <= 0;
    }

    @Override
    protected void doApprove(
        ExpenseRequest request
    ) {

        System.out.println(
            "Approved by Director"
        );
    }
}

```

Build the Approval Chain

```
ExpenseApprover teamLead = new TeamLeadApprover();
```

```
ExpenseApprover manager = new ManagerApprover();
```

```
ExpenseApprover director = new DirectorApprover();
```

```

teamLead
    .setNext(manager)
    .setNext(director);

```

Now:

```

teamLead.approve(
    new ExpenseRequest(

```

```
        new BigDecimal("1500")
    )
);
```

Output:

Approved by Manager

Lesson 16 — Chain Types

There are two common forms.

Type 1 — Every Handler Processes

Example:

Request -> Logging ->Authentication ->Authorization ->Validation ->Business

Every handler does something.

This is common in:

Filters

Middleware

Interceptors

Security

Type 2 — First Matching Handler Processes

Example:

Expense

↓

Team Lead?

↓ no
Manager?
↓ yes
STOP

This is common in:

Approval
Support Escalation
Exception Handling
Command Routing

We should understand both.

Lesson 17 — SOLID Connection

Single Responsibility Principle

Each handler performs one responsibility.

ValidationHandler => Validation
AuthenticationHandler => Authentication
AuthorizationHandler => Authorization
FraudHandler => Fraud Check

Open/Closed Principle

Need blacklist checking?

Create:

```
public class BlacklistHandler  
    extends BasePaymentHandler {
```

```
}
```

Insert it into the chain.

Existing handler implementations remain unchanged.

Dependency Inversion Principle

High-level processing can depend on:

PaymentHandler

instead of concrete checks.

Lesson 18 — Dynamic Chain

A powerful advantage is runtime configuration.

Suppose fraud checking is optional.

```
List<PaymentHandler> handlers = new ArrayList<>();
```

```
handlers.add(  
    new ValidationHandler()  
);
```

```
handlers.add(  
    new AuthenticationHandler()  
);
```

```
if (fraudCheckEnabled) {
```

```
    handlers.add(  
        new FraudHandler()  
    );
```

```
}
```

```
handlers.add(  
    new ProcessPaymentHandler()  
);
```

We can build different chains for different environments.

Lesson 19 — Enterprise Example

Imagine an API request:

HTTP Request

|

Request ID

|

Logging

|

Authentication

|

Authorization

|

Rate Limit

|

Validation

|

Controller

This is one reason Chain of Responsibility appears naturally in web frameworks.

We are already using the idea.

Lesson 20 — Chain vs Decorator

Students just learned Decorator, so compare them immediately.

Decorator

Metrics -> Logging -> PaymentGateway

Purpose:

Add behavior around one object.

Every decorator usually delegates.

Chain of Responsibility

Validation -> Authentication -> Authorization -> Payment

Purpose:

Pass a request through handlers.

A handler may:

- Continue
- Stop
- Reject
- Handle completely

Easy Difference

Decorator wraps behavior. Chain passes responsibility.

Lesson 21 — Chain vs Strategy

Strategy:

Payment

↓

Choose one strategy

ABA

or

Wing

or

Stripe

One behavior selected.

Chain:

Validation

↓

Security

↓

Fraud

↓

Payment

Multiple handlers may participate.

Common Mistakes

Mistake 1 — One Handler Does Everything

Bad:

```
public class ValidationHandler {
```

```
void validate();  
void authenticate();  
void authorize();  
void checkFraud();  
void rateLimit();  
}
```

That defeats the purpose.

Mistake 2 — Hard-Coding Next Handler Inside Handler

Bad:

```
public class ValidationHandler {  
  
    private final AuthenticationHandler next =  
        new AuthenticationHandler();  
}
```

Now the chain cannot be reconfigured.

Better:

```
setNext(...)
```

or inject the chain externally.

Mistake 3 — Hidden Execution Order

If order matters:

```
Authentication  
must happen before  
Authorization
```

make the chain configuration obvious.

Mistake 4 — Continuing After Failure

Bad:

```
if (!valid) {  
    log("Invalid");  
}
```

```
next(context);
```

The request still continues.

If invalid, stop it.

Mistake 5 — Using Chain for One Simple Check

If we have:

Validate Email

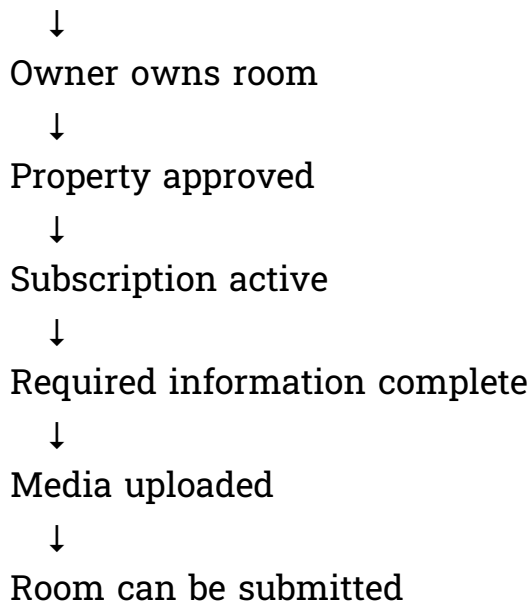
only one check, a chain is unnecessary.

Homework — Room Publishing Validation Chain

Build a Room Publishing validation pipeline.

Before a room can be submitted:

Room exists



Create:

RoomExistsHandler
OwnershipHandler
PropertyStatusHandler
SubscriptionHandler
RoomCompletenessHandler
MediaHandler
SubmitRoomHandler

The client should call only:

```
roomPublishingChain.handle(context);
```

Homework Requirements

1. Create RoomPublishingContext.
2. Create RoomPublishingHandler.
3. Create base handler.
4. Implement all validation handlers.

5. Stop when one validation fails.
6. Submit room only when all handlers pass.
7. Add `ModerationEnabledHandler` without changing existing handlers.
8. Build one chain for production.
9. Build a smaller chain for local development.
10. Explain SRP and OCP.

Interview Questions — Class 1

1. What problem does Chain of Responsibility solve?
2. What are the main participants?
3. How does a handler pass a request to the next handler?
4. Can a handler stop the chain?
5. Can every handler process the same request?
6. Can only one handler process a request?
7. What is the difference between Chain and Decorator?
8. What is the difference between Chain and Strategy?
9. How does Chain support OCP?
10. What happens if no handler processes the request?
11. Why should next-handler wiring happen outside handlers?
12. Give three enterprise examples.